

# Two ways to be a value

Algebraic data types and objects

---

SW7 · Aalborg University Copenhagen · Autumn 2026

# The question

What is a datatype? What is an object? What is an algebraic data type?

# What is a datatype?

Some answers we would all give:

- `int` — a set of numbers?
- `int32` — the same, but exactly  $2^{32}$  of them?
- `struct` — several values glued together?
- `class` — values with code attached?
- “a collection of values” — of what, arranged how?

Each describes one aspect. The question returns four times this semester:

|            |                                            |
|------------|--------------------------------------------|
| S3 • today | how a datatype <b>organises</b> its values |
|------------|--------------------------------------------|

|    |                                         |
|----|-----------------------------------------|
| S9 | what a type <b>claims</b> about a value |
|----|-----------------------------------------|

|     |                          |
|-----|--------------------------|
| S11 | what a type <b>hides</b> |
|-----|--------------------------|

|     |                             |
|-----|-----------------------------|
| S12 | a type as a <b>relation</b> |
|-----|-----------------------------|

# What is an object?

In everyday use, “object” bundles several design decisions. Five of them, each treated on its own day:

| <b>bundled into “object”</b> | <b>actually a separate axis</b>                 |                 |
|------------------------------|-------------------------------------------------|-----------------|
| mutable fields, identity     | state vs. persistence                           | S2, S7          |
| inheritance, extends         | subtyping                                       | S10             |
| classes                      | classes vs. prototypes (Self, JS)               | today, one line |
| declared implements          | nominal vs. structural typing (Java vs. Go)     | S10             |
| static types                 | Smalltalk has none — the purest object language | S9              |

In this session, an object is a **value used only through the operations it carries**.

State and identity can be added — and usually are — but they are not essential.

# What is an algebraic data type?

Last week we read two of them without the name — and used the first in the type of `pop`:

```
type 'a option = None | Some of 'a
type 'a list   = []    | (::) of 'a * 'a list
```

```
val pop : 'a t -> ('a * 'a t) option
(* the value and the new version – or nothing *)
```

Every value is built from a **closed list of shapes**, and taken apart the same way.

By the end of this first hour: why a closed list of shapes is the **natural design** for many datatypes — and what the opposite design looks like.

# Two ways to be a set

The discovery, in JavaScript

# One interface

A set of integers. Four operations, and nothing else:

```
empty           // the set with no elements  
insert(s, n)    // the set s, with n added  
contains(s, x)  // is x in s?  
union(s, t)     // every element of s, or of t
```

Client code uses only these four names.

Two implementations follow. The client cannot tell them apart. At least at first.

# Way 1 — a set **is data**

```
const empty1      = [];  
const insert1     = (s, n) => s.includes(n) ? s : [...s, n];  
const contains1   = (s, x) => s.includes(x);  
const union1      = (s, t) => [...s, ...t.filter(x => !s.includes(x))];
```

Every operation **inspects**<sup>\*</sup> the array.

`union1` inspects **both** of its arguments.

<sup>\*</sup>inspect: read the representation directly — here as an array; in OCaml, as a constructor and its arguments.

## Way 2 — a set **is a function**

A set is a function that answers one question: *is x inside?*

```
const empty2    = x => false;
const insert2   = (s, n) => (x => x === n || s(x));
const contains2 = (s, x) => s(x);
const union2    = (s, t) => (x => s(x) || t(x));
```

Every operation **calls**<sup>\*</sup> the set. `contains2` is just the call.

`union2` can do nothing with `s` and `t` except call them. **There is no inside.**

<sup>\*</sup>call: use a value only through the interface it exposes, learning nothing about how it is made.

# Two inserts, unfolded

## Way 1

```
const s3  = insert1(empty1, 3)
      // = [3]
const s35 = insert1(s3, 5)
      // = [3, 5]
```

Each step produces **data**: an array anyone may inspect.

## Way 2

```
const s3  = insert2(empty2, 3)
      // = x => x === 3 || empty2(x)
const s35 = insert2(s3, 5)
      // = y => y === 5 || s3(y)
```

Each step produces a **closure**<sup>\*</sup>. s35 holds s3; nothing runs until someone calls it.

Both s35 are chains of length two: [3, 5], and a function holding 5 and s3, which holds 3 and empty2. Same shape — and, from the operations' side, one can be inspected while the other can only be called.

<sup>\*</sup> closure: a function together with the bindings it captured where it was defined — here n and the previous set.

# Same client, same answers

```
const a1 = insert1(insert1(empty1, 3), 5), b1 = insert1(empty1, 4);  
const a2 = insert2(insert2(empty2, 3), 5), b2 = insert2(empty2, 4);  
const test1 = [3, 4, 5, 6].map(x => contains1(union1(a1, b1), x));  
const test2 = [3, 4, 5, 6].map(x => contains2(union2(a2, b2), x));
```

Run

```
way 1: [true, true, true, false]  
way 2: [true, true, true, false]
```

Nothing so far tells the two apart. But what if we **extend** the interface with

1. a new operation
2. a new kind of set
3. an operation on two sets

# Extension 1 — a new operation: size

## Way 1

```
const size1 = s => s.length;
```

One line. Nothing else is touched.

Run

```
size1(union1(a1, b1)) = 3
```

## Way 2

```
const size2 = s => ???
```

A function cannot be asked how many inputs it accepts.

... unless a set carries **two** functions:

```
const insert2b = (s, n) => ({  
  contains: x => x === n || s.contains(x),  
  size:    () => s.size() + 1 });
```

So a set becomes a **record of operations** sharing hidden data — and every existing set must be rebuilt this way.

# Extension 2 — a new kind of set: the even numbers

## Way 1

No array holds the even numbers. There are now **two kinds** of set, told apart by a tag:

```
{ kind: 'finite', xs: [3, 5] } // as before
{ kind: 'evens' }           // no array
```

```
const contains1 = (s, x) =>
  s.kind === 'evens' ? x % 2 === 0
                    : s.xs.includes(x);
const union1 = (s, t) => /* four cases */
```

— and **every** operation now begins by **reading the tag**.

## Way 2

```
const evens = x => x % 2 === 0;
const test3 = [3, 4, 5, 6]
  .map(x => contains2(union2(a2, evens), x));
```

One line — an *infinite* set — and it unions with the others as if nothing happened.

Run

```
union2(a2, evens): [true, true, true, true]
```

# Extension 3 — an operation on two sets

## Way 1

```
const union1 = (s, t) =>
  [...s, ...t.filter(x => !s.includes(x))];

const equal1 = (s, t) =>
  s.length === t.length &&
  s.every(x => t.includes(x));
```

Inspects both. Could sort, merge, deduplicate, compare.

## Way 2

```
const union2 = (s, t) => x => s(x) || t(x);

const equal2 = (s, t) => ???
```

Can only call. Enough for union —

**not enough** for equal: no finite test decides whether two functions agree on every input.

**Way 1:** an operation on two sets sees both representations — it can merge them, or compare them.

**Way 2:** it can only ask each set about an  $x$  and combine the answers. The two sets themselves are **never compared**.

# Two ways, stated

|                          | Way 1 — a set is data   | Way 2 — a set is a function |
|--------------------------|-------------------------|-----------------------------|
| a new operation          | one new function        | every set rebuilt           |
| a new kind of set        | every operation changed | one new function            |
| an operation on two sets | inspect both            | call both — no comparison   |

**Way 1:** a value **is** one of a closed collection of representations, open to every operation. A new operation is free; a new representation affects every operation.

**Way 2:** a value **is** its own operations, bundled together when it is made. A new implementation is free; a new operation affects every implementation.

Two ways of **implementing one interface**. Neither is a language feature — we used arrays and arrow functions. The names come next.

# Algebraic data types

Way 1, made precise

# Naming the two ways

## what we built

**Way 1** — a value is data; operations inspect it

**Way 2** — a value is a function; operations call it

## what it is called

**algebraic data types**

**objects**

Both are older than the languages we write them in. Reynolds (1975) called them *user-defined types* and *procedural data structures* — “**complementary approaches to data abstraction**”.

Two ADTs, one acronym.

**Abstract** data type (Liskov & Zilles, 1974): a type its clients know only through its operations — session 11.

**Algebraic** data type (Hope, 1980): a type whose values come in a closed list of shapes — today.

# An enumeration

```
type note = C | D | E | F | G | A | B
```

```
let semitones n = match n with  
  | C -> 0 | D -> 2 | E -> 4 | F -> 5 | G -> 7 | A -> 9 | B -> 11
```

Reading rule: | is **or**. A note is C, or D, or ...

The constructors are values we **invent**. C is not 0; nothing is inside it. It is a **tag**, as kind: 'evens' was for the set — but one of exactly seven values of a new type, *note*, each distinct from the others by construction.

The usual encoding in C, and in Java before `enum`, was `static final int C = 0, D = 2, ...`  
Now `C + D` compiles, and 7 is accepted wherever a note is expected. A variant type allows neither.

# Carrying values: products and sums

```
type pitch      = { note : note; octave : int }  
type duration = Whole | Half | Quarter | Eighth  
type symbol    = Note of duration * pitch | Rest of duration
```

- `pitch` is a **product** type: each of its values holds every field.
- `symbol` is a **sum** type: each of its values is exactly one alternative — a `Note` carrying a duration and a pitch, or a `Rest` carrying a duration. That is what `of` reads as.

A `Rest` carries no pitch: no field is left empty or set to `null` — a rest with a pitch cannot be written at all.

```
let beats s = match s with  
  | Note (d, _) | Rest d -> beats_of d
```

# Why “algebraic”

Count the values of a type

|                                                       |                                                      |
|-------------------------------------------------------|------------------------------------------------------|
| <code>unit</code>                                     | <b>1</b>                                             |
| <code>bool</code>                                     | <b>2</b>                                             |
| <code>note</code>                                     | <b>7</b>                                             |
| <code>duration</code>                                 | <b>4</b>                                             |
| <code>pitch = note × int</code>                       | <b><math>7 \cdot  \text{int} </math></b>             |
| <code>symbol = duration × pitch +<br/>duration</code> | <b><math>4 \cdot 7 \cdot  \text{int}  + 4</math></b> |

The type constructors **form an algebra**. Counting is its simplest model — and it works for infinite types too, as laws rather than numbers.

Products multiply, sums add

$$|A \times B| = |A| \cdot |B| \quad |A + B| = |A| + |B|$$

The laws of arithmetic hold as **isomorphisms** of types, with a conversion function each way:

$$A \times (B + C) \cong A \times B + A \times C$$

# Two encodings of one type

## A struct with a tag

```
struct symbol {  
    int tag;          /* 0 = note, 1 = rest */  
    int duration, note, octave;  
};
```

$2^{128}$  states. Most mean nothing: a rest with a pitch; a tag of 7.

## A sum

```
type symbol =  
    | Note of duration * pitch  
    | Rest of duration  
    (* two shapes, nothing else *)
```

$4 \cdot 7 \cdot |\text{int}| + 4$  states. Each means something.

Sums do not add possibilities. They **remove meaningless ones**.

**option** absence as **one explicit case**: None | Some of 'a has one more value than 'a, and every match must handle it.

**null** absence added to **every type at once** and recorded in none: no match is required, but the missing case surfaces at runtime as a `NullPointerException`.

# Recursion: a type mentioned in its own definition

```
type 'a list = [] | (::) of 'a * 'a list
```

```
type set = Empty | Insert of int * set      (* slide 6, as a type *)
```

```
let rec contains s x = match s with  
  | Empty          -> false  
  | Insert (n, rest) -> n = x || contains rest x
```

Its equation, solved:  $L = 1 + aL \Rightarrow L = \frac{1}{1-a} = 1 + a + a^2 + a^3 + \dots$

A list is empty, or one element, or two, or three ... — **a tuple of some length**, one shape for each length.

The set we wrote as an array on slide 6 is now a type. The closed list of shapes — Empty, Insert — is written down **where the compiler can read it**.

# Constructors\* are functions — but not ordinary ones

```
length : 'a list -> int           (* an ordinary function *)
```

```
Note   : duration * pitch -> symbol (* a constructor *)
```

An ordinary function may **lose information**: from `length xs` we cannot recover `xs`; many lists share one `length`.

A constructor **loses no information**. Three properties no ordinary function has:

|                  |                                           |
|------------------|-------------------------------------------|
| <b>injective</b> | different arguments give different values |
|------------------|-------------------------------------------|

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <b>disjoint</b> | different constructors never build the same value |
|-----------------|---------------------------------------------------|

|                   |                                                    |
|-------------------|----------------------------------------------------|
| <b>exhaustive</b> | every value of the type came from some constructor |
|-------------------|----------------------------------------------------|

So every value was built in **exactly one way**: one constructor, applied to the values it carries.

\* Unlike Java's constructors, these run no code and can be taken apart: a tag with a payload.

# Matching is the inverse

```
let s = Note (d, p)           (* forwards: build *)
match s with Note (d, p) -> ... (* backwards: recover d and p *)
```

A constructor goes from the carried values to the value:  $(d, p) \mapsto \text{Note } (d, p)$ .

A match goes **from the value back to the carried values**:  $\text{Note } (d, p) \mapsto (d, p)$ .

It can, because every value was built in exactly one way.

- We cannot match on  $f \ x$  for an arbitrary  $f$ : there is no inverse to run.
- We cannot match on a Way 2 value: any function may have produced it, and nothing in it says which, or from what — its environment is sealed, and there is no fixed list of makers.  
**There is nothing to invert.**
- The list of shapes used to live in a comment. Now it lives in the type, and **the compiler reads it**: every match covers every shape, and no case is unreachable.

# Algebraic data types: a timeline

1970s    Pascal variant records; C `union` with a tag field — the list of shapes is a convention, unchecked

---

1980    **Hope** (Burstall, MacQueen, Sannella): algebraic data types with pattern matching

---

1984 → Standard ML, then OCaml and Haskell: the ML family makes them the default

---

2014 → Swift enums, Rust enums, Kotlin sealed classes, TypeScript discriminated unions

---

2021 → Python 3.10 `match`; Java 17 `sealed`, Java 21 `pattern switch`

“Algebraic” also names the mathematics: the values of the type are **exactly the terms** built from its constructors, nothing more and nothing identified — the *initial algebra* of its constructors (Goguen, Thatcher, Wagner, Wright, 1977).

# Objects

Way 2, made precise

# The set as objects

```
type set =  
  < contains : int -> bool; size : int >  
  
let empty : set = object  
  method contains _ = false  
  method size = 0  
end  
  
let insert (s : set) n : set = object  
  method contains x = x = n || s#contains x  
  method size = s#size + 1  
end
```

type set is **the interface** of a set — extension 1's record of operations, as a type — and under it two kinds of set, `empty` and `insert`.

In the Way 1 type they were the constructors `Empty` and `Insert`; here the type lists only **operations**, and the kinds live in the code.

**No match on a set:** nothing at compile time says how many kinds exist, or which one we hold.  
**That is the point:** which kind we hold is decided at run time, by the object, when it is called — **dynamic dispatch**.

\* `object ... end` builds one object and `#` calls a method. No class is involved; OCaml does not need one.

# The same, in Java

```
abstract class Set {
    abstract boolean contains(int x);
    abstract int size();
}
class Empty extends Set {
    boolean contains(int x) { return false; }
    int size() { return 0; }
}
class Insert extends Set {
    private final Set s; private final int n;
    Insert(Set s, int n)
    { this.s = s; this.n = n; }
    boolean contains(int x)
    { return x == n || s.contains(x); }
    int size() { return s.size() + 1; }
}
```

The abstract class is the interface<sup>\*</sup>; each subclass is one kind, with its own operations.

The fields are private<sup>\*\*</sup>: from a client's side, a Set is only what its methods answer.

<sup>\*</sup> A Java interface says the same, without fields or shared code.

<sup>\*\*</sup> **private**: no client reads the field. **final**: assigned once, in the constructor — a set never changes after it is made.

# A new kind, in Java

A new kind is one more class:

```
class Evens extends Set {  
    boolean contains(int x) { return x % 2 == 0; }  
    int size()           { return Integer.MAX_VALUE; }  
}
```

Set, Empty, Insert and every client **stay as they are**:

```
Set s = new Insert(new Insert(new Empty(), 3), 5);  
s.contains(4);
```

Which contains runs is decided by the object `s` holds, at run time — three calls down the chain, none visible to the caller.

# Classes

Nothing so far needed a class. In everyday languages, a class bundles several jobs:

- **factory** — new makes objects
- **code sharing** — methods written once
- **type** — Set in signatures
- **privacy boundary**
- **inheritance** — session 10
- **namespace** — Set.empty

Languages disagree on which to bundle, and on whether to have a class at all:

Java

Smalltalk

JavaScript

OCaml

Go

```
public boolean equals(Object o) {  
    return o instanceof Insert i && i.n == n && i.s.equals(s);  
}
```

All six jobs in one box. Privacy is **per class**, not per object: equals reads another object's n. The static methods of a class are the operations of an abstract data type.

# Abstract data types versus objects

What each hides, and how each grows — Reynolds 1975 · Cook 2009 · Wadler 1998

# One abstraction barrier, placed two ways

Both ways hide how an operation works: a client calls `contains` and gets an answer, never seeing the body — the array scan, or the chain of calls. Any function does this much. What each way adds differs:

**[ADT]** Way 1, behind a wall we will build in session 11. The **representation type** is hidden: behind an `.mli`, as session 2's stack, `type t` shows no constructors, so no client can match on a set or build one by hand. Cook calls this **type abstraction**. Only a compiler can enforce it.

**[Objects]** Way 2, as it stands. Nothing is hidden at the type level: the interface type is public. What is hidden is the environment of one closure — this `s`, this `n` — and it is hidden by being a function: nothing but a call can reach in. Cook calls this **procedural abstraction**: a procedure's wall, with the data inside.

One wall around the whole type, or **one wall around each value**.  
Objects need no type system to keep their secrets; abstract data types do.

# Complementary approaches to data abstraction

|                             | abstract data type                              | object                                            |
|-----------------------------|-------------------------------------------------|---------------------------------------------------|
| what is abstracted          | the representation <b>type</b>                  | the <b>procedures</b> in a value                  |
| operation bodies            | hidden                                          | hidden                                            |
| representation              | one, hidden                                     | none in the interface                             |
| an operation is written     | once, for every value                           | by each value, for itself                         |
| sees another value's inside | yes — <code>union1</code> , <code>equal1</code> | no — <code>union2</code> ; no <code>equal2</code> |
| implementations             | one per type; two do not mix                    | any number, one type; they mix                    |

Reynolds (1975) saw the two as **complementary**; Cook (2009) gave the table its rows.

Neither is a weaker form of the other. An object is not an abstract data type missing its type: it hides its data another way. An abstract data type is not an object missing dispatch: every value of `t` runs the same contains; there is nothing to choose at run time.

**They hide different things, by different means** — the compiler, or the closure.

# The matrix: kinds × operations

|        | contains | size | union | equal |
|--------|----------|------|-------|-------|
| Empty  | ✓        | ✓    | ✓     | —     |
| Insert | ✓        | ✓    | ✓     | —     |
| Evens  | ✓        | ✓    | ✓     | ?     |

**Algebraic data types** (Way 1): one function per operation, every kind inside it — a **column** at a time.

One more operation is one more column, `equal` included: a function inspects both sets.

**Objects** (Way 2): one class per kind, every operation inside it — a **row** at a time.

One more kind is one more row, `evens` included. `equal` gets no row: a row cannot inspect another.

Wadler (1998) named the problem of growing both ways at once, with the compiler still checking every cell: **the expression problem**. Its resolutions are session 10.

# One language, both ways

## Way 2 — abstract class and subclasses

```
abstract class Set { ... }  
class Empty extends Set { ... }  
class Insert extends Set { ... }  
class Evens extends Set { ... }
```

Rows. The part before, as written.

## Way 1 — sealed, records, switch

```
sealed interface Shape  
  permits Circle, Square, Tri {}  
record Circle(double r) implements Shape {}  
record Square(double s) implements Shape {}  
record Tri(double b, double h) implements Shape {}  
  
static double area(Shape sh) {  
  return switch (sh) {  
    case Circle c -> Math.PI * c.r() * c.r();  
    case Square s -> s.s() * s.s();  
  };  
}
```

permits is |, record is of, switch is match. Java required a quarter of a century, and three additions to the language, before it could express the second form. That delay is the measure of its **default**.

# Exhaustiveness in three languages

One case left out of a match, on purpose. Three compilers, three answers:

OCaml

```
Warning 8 [partial-match]: this pattern-matching is not exhaustive.  
Here is an example of a case that is not matched: Repeat (_, _)
```

Java 21+

```
A.java:10: error: the switch expression does not cover all possible input values
```

Python 3.10+

```
square: 9  
triangle: None
```

OCaml warns and **names a missing value**. Java refuses to compile. Python runs, and the missing case becomes a None that travels on.

OCaml: the lab's melody with Repeat added and no match updated. Java, Python: Tri left out of area.

# The checks are dual

## Way 1 — by column

```
Warning 8 [partial-match]:  
this pattern-matching is not exhaustive.  
Here is an example of a case that  
is not matched: Repeat (_, _)
```

Does every operation handle every kind?

## Way 2 — by row

```
B.java:9: error: Evens is not abstract  
and does not override abstract method  
size() in Set
```

Does every kind provide every operation?

Each check is blind where the other sees. Extension is blocked exactly where the **declaration site belongs to someone else**: the permits list, or the abstract class.

An instanceof chain is not a match: nothing checks it — hence the visitor.

# Visitor: pattern matching by hand

```
interface Visitor<R> {
    R onEmpty();
    R onInsert(Set s, int n);
}
abstract class Set {
    abstract <R> R accept(Visitor<R> v);
}
class Empty extends Set {
    <R> R accept(Visitor<R> v)
    { return v.onEmpty(); }
}
class Insert extends Set {
    <R> R accept(Visitor<R> v)
    { return v.onInsert(s, n); }
}
```

Way 1 would write one match, an arm per kind. Java has none; we build it by hand:

- **the visitor**: one method per kind, `onEmpty` and `onInsert` — the arms
- **accept**: each kind's one new method; it calls its own arm and hands over what it carries
- `s.accept(v)` **dispatches twice**: the set picks `accept`, `accept` picks the arm. **The set chose the arm, not the caller.**

Way 1's trade, cost included: size is one visitor, written outside; a new kind needs `onEmpty` in every visitor.

`Empty | Insert of int * set ≅ { on_empty; on_insert }`: a sum of products is a product of functions.

# What each way is good for

## Objects

- **extension by strangers** — a framework defines `Widget`; plug-ins add kinds it never sees
- **state that changes, behind a wall** — a counter, a connection: only its own methods touch it (Parnas, 1972)
- **collections sharing only an interface** — a list of sets, of any kinds

## Algebraic data types

- **closed domains, modelled exactly** — messages, UI states, `Option`
- **programs that process trees** — compilers, interpreters, query planners, JSON: few node kinds, many operations, some on two nodes at once
- **code that grows by a kind** — the compiler lists every site to fix
- **data that travels** — files, the network: only a tag and its payload can go

Much “object” code is data with getters — the shape that `record` and `sealed` were added to Java to fix.

# Conclusion

Language defaults · the question in later sessions · the lab's type

# Every language chooses a default

| language             | default                 | the other way                                                     |
|----------------------|-------------------------|-------------------------------------------------------------------|
| OCaml                | <b>Way 1</b>            | <code>object ... end</code> — rarely used                         |
| Rust, Swift, Haskell | <b>Way 1</b>            | <code>dyn Trait</code> , protocols, type classes                  |
| Java                 | <b>Way 2</b>            | <code>sealed + record + switch</code> , since 2021                |
| Smalltalk            | <b>Way 2</b> only       | no static list of kinds — session 9                               |
| Python, JavaScript   | <b>Way 2</b> , untyped  | <code>match</code> , unchecked; a <code>kind</code> field by hand |
| Go                   | <b>Way 2</b> , by shape | none: a <code>switch</code> on constants, unchecked               |

Paradigm  $\neq$  language: a language only makes some choices the default. The friction of saying the other thing is the **evidence of the default**.

# The same question in sessions 8, 9, 11 and 12

This session answered *what is a datatype?* by how it **organises** its values: by construction, or by behaviour. Four later sessions answer it again:

S8 a type as a **protocol**: which messages, in which order — session types

---

S9 what a type **claims** about a value: a type as a proof; Smalltalk's one big sum

---

S11 what a type **hides**: abstract data types. Way 1 behind a wall; session 2's `.mli` returns

---

S12 a type as a **relation**: tables, queries; a query plan is a tree — a Way 1 program

# Lab 3: the melody type

```
type melody =  
  | Note of duration * pitch  
  | Rest of duration  
  | Seq of melody * melody    (* after *)  
  | Par of melody * melody    (* together *)
```

Every function in the lab is one match on this type. `melody.ml` has the holes; `dune exec ./main.exe` reports each one.

- |      |                                                                                                             |
|------|-------------------------------------------------------------------------------------------------------------|
| A    | <code>count_notes</code> , <code>length_in_beats</code> : Seq adds, Par takes the maximum                   |
| B    | <code>to_events</code> , then a <code>.wav</code> of Bach's Invention in C; add Repeat, follow the warnings |
| C    | <code>transpose_octave</code> , <code>pretty</code> , <code>retrograde</code> , <code>simplify</code>       |
| read | CS3110 chapter 3: sections 3.1, 3.5, 3.6, 3.8, 3.11, 3.12                                                   |